

Electronic Companion

Accepted Service Adaptation:

Exact Parametric Quotients and Minimal Additional Writable Memory

Anonymous manuscript for Operations Research

Revision R36, September 24, 2026

EC.1 The Laminar Polymatroid Connection

This section supplies the feasible-set link to Federgruen and Groenevelt (1986); Groenevelt (1991); it is not offered as a new polymatroid theorem. In the variables of Proposition 2.1, set $\ell_h = w_h a_{v(h)} l_{v(h)}$, $u_h = z_h - \ell_h$, and $c_h = w_h a_{v(h)} (h_{v(h)} - l_{v(h)})$. The residual subtree capacities are $C_h = w_h B_{v(h)} - \sum_{j \succeq h} \ell_j \geq 0$, and the residual total is $b' = b_0 - \sum_h \ell_h \geq 0$. Write $\mathcal{T}_h$ for the descendant set at h . For $A \subseteq \mathcal{T}_h$, define the rank recursively by

$$f_h(A) = \min \left\{ C_h, c_h \mathbf{1}_{\{h \in A\}} + \sum_{j \text{ child of } h} f_j(A \cap \mathcal{T}_j) \right\}. \quad (1)$$

The root function f_0 is normalized, nondecreasing, and submodular. Indeed, the expression inside the minimum is a sum of a nonnegative modular function and submodular functions on disjoint ground sets; truncation at a nonnegative constant preserves nondecreasing submodularity. The latter fact follows directly from diminishing marginal increments: the increment of a truncated function is its original nonnegative increment capped by the remaining distance to the truncation level, both of which cannot increase when the conditioning set grows.

The residual packing region is exactly $P(f_0) = \{u \geq 0 : \sum_{h \in A} u_h \leq f_0(A) \text{ for every } A\}$. To verify the rank, maximize allocation to a subset A on a subtree. The independent child capacities contribute their ranks and the current coordinate contributes $c_h \mathbf{1}_{\{h \in A\}}$; the current cap truncates their sum. Any value between zero and that sum can be obtained by scaling a feasible nonnegative allocation, which proves (1). Conversely, the subset inequalities for singleton coordinates and for descendant sets imply the original box and subtree rows, since their ranks do not exceed the respective capacities. This proves equality of the regions.

Feasibility gives $b' \leq f_0(\mathcal{T})$. Define $f^{b'}(A) = \min\{b', f_0(A)\}$. The exact-payment feasible set is the base

$$\left\{ u \in P(f^{b'}) : \sum_h u_h = f^{b'}(\mathcal{T}) = b' \right\}.$$

Truncation matters when the promised total is below the maximum feasible allocation. The objective after translation remains separable strictly concave. Hence both the laminar and the polymatroid connections are exact, including nonzero local lower bounds and a root equality. Our input-compression and execution-memory claims concern this repeated-subtree subclass, not separable allocation over all polymatroids.

EC.2 Reading Guide and Preservation of the General Theory

The main manuscript proves the laminar equivalence, the occurrence quotient, rational preprocessing bounds, minimal execution alphabet, heterogeneous frontier, dispersion comparison, and fixed-table supporting inequality. This companion supplies implementation details, additional boundary arguments, and reproducibility specifications. It is not a substitute for the central proofs.

The R30 manuscript, electronic companion, and computational package remain preserved in their revision directory, and all earlier revision directories remain unchanged. The provenance files map inherited results to their original locations. This organization keeps R31 centered on tight parametric representation and writable-memory conclusions without deleting the broader accepted-adaptation theory. In particular, the finite-price assumptions do not retroactively restrict the older convex, switching, promise-state, or certificate theorems.

The retained main manuscript contains accepted balance conditions, continuation-price transfers, release frontiers, comparator-adjusted continuation rents, switching-friction paths, information hierarchies, promise-state recursion, continuous-state envelopes, extensive-form-dual shared-table representability, reward-only and correlated cache certificates, and cache governance. Its companion retains stock-identification refinements, detailed proofs, active-set and dual checks, continuous-state arguments, and adaptive deployment bounds. Historical and computational supplements remain separate readable volumes. These materials support reuse of the general institution while the new theorems state their own narrower computational hypotheses explicitly.

EC.3 Coefficient-Event Compilation

For each response, store ordered rational knots $k_1 < \dots < k_s$ and one affine coefficient pair (m_i, c_i) on each interval, including the two tails. Equal adjacent pairs are removed. Evaluation uses binary search and the convention that a knot belongs to its right interval; continuity makes the value independent of that convention.

At vertex v , initialize the left-tail coefficient with $a_v h_v + \beta \sum_w p_{vw} U_w$. The local clipped payment changes slope and intercept at

$$(r_v - q_v h_v)/a_v, \quad (r_v - q_v l_v)/a_v.$$

For each successor knot, add the weighted difference between its new and old coefficient pair to the event at that knot. Combining equal-key events before sorting ensures that a repeated downstream breakpoint is stored once at the current vertex. Starting from the left-tail pair, sweep events to recover the pre-cap lines. This operation merges coefficients; it never recovers a slope from two evaluations at newly generated cap crossings.

If the cap is redundant, store the pre-cap curve. Otherwise find the first crossing of B_v by scanning the nonconstant affine pieces. Replace the entire prefix by the constant B_v , retain the original crossing line to its right, and discard any newly equal neighboring pairs. The crossing may coincide with an existing knot or lie

at the left endpoint of a constant equality plateau. Both cases use the same continuity convention.

The input checker requires rational integers or strings, positive payment and quadratic coefficients, a valid physical interval, positive outgoing probabilities summing to one, topological vertex order, and reachability from the root. Duplicate edges must first be aggregated. It rejects an empty feasible continuation interval or an out-of-range root promise rather than returning an approximate feasible answer. Floating-point primitives are rejected by the exact compiler because their intended rational interpretation would otherwise be ambiguous.

EC.3.1 A direct plateau argument

Let x_1, x_2 maximize $G(x) - \eta_1 Y(x)$ and $G(x) - \eta_2 Y(x)$ over the same compact convex pre-cap set, with $\eta_1 < \eta_2$. Optimality gives

$$G(x_1) - G(x_2) \geq \eta_1(Y(x_1) - Y(x_2)), \quad G(x_1) - G(x_2) \leq \eta_2(Y(x_1) - Y(x_2)).$$

If the two payments coincide, these inequalities imply equal rewards. Both actions then maximize both objectives. Strict concavity forces $x_1 = x_2$. A payment plateau therefore changes neither the optimal local tier nor any future tier, although the price supporting that plan may vary. This proves policy uniqueness at a cap-equality plateau without assuming uniqueness of a Lagrange multiplier.

At a response's upper tail, the implementation returns one finite price strictly below its first knot. At the lower tail, it returns an endpoint of the last decreasing segment; a completely constant response uses price zero. None of these choices changes the optimum. If the root itself is capped exactly at its promised payment, its cap multiplier and equality price may be redistributed, but their contribution to the objective remains the same because the two right-hand sides agree.

EC.3.2 Explicit pseudocode

The following procedure is equivalent to the executable coefficient-event implementation.

Process vertices in reverse topological order. At each vertex, validate the feasible lower payment. Create the two local clipping events when the interval is nondegenerate. Add discounted probability-weighted successor coefficient changes into a common event dictionary. Sort its keys and sweep to obtain the pre-cap curve. When necessary, invert that curve at the current cap and replace its left prefix by a constant. Store the remaining lines and current barrier. After the root query arrives, invert the root curve at the exact promised payment. Propagate the maximum of the incoming price and current barrier, aggregate identical reached vertex-price pairs, and record actions and multipliers. Finally minimize the reached execution graph by backward behavioral signatures.

The procedure constructs all graph response curves, but it constructs execution states only for the supplied query. A different query may change both the reached pairs and the minimized machine. The table-storage bound and writable-alphabet bound describe different data structures.

EC.4 Bit Bounds for Execution and Certificates

The main bit theorem deliberately concerns response preprocessing and one inversion. The next observation explains why exact execution remains polynomial while a printed total value can be much longer than an individual response coefficient.

Proposition EC.4.1 (A conservative execution-height bound). *Let S be the bit-height bound in Theorem 3.4, with the root query included in the input length. For a fixed query, all rational numbers needed by a compact execution certificate, including its aggregate reward and dual value, have bit height $O(NS + NL + \log(N + 1))$ under a common-denominator implementation. Exact forward/backward execution and auditing thus admit polynomial bit complexity on at most $N(N + 1)$ reached pairs and $M(N + 1)$ transitions.*

Proof. The discounted weight of a reached pair is a sum of probabilities of public paths that produce its price label. Each path contains at most $N - 1$ edges. The same product-of-input-denominators argument as in Theorem 3.4 bounds its rational denominator by D_0^{N+4} and its numerator by a comparable height, since its weight is at most one. This representation does not multiply the denominators of individual histories.

There are at most $N + 1$ distinct numerical prices. A local action is either a physical endpoint or $(r_v - a_v s)/q_v$ for one such price s . A local reward has degree at most two in that price, with rational primitive coefficients. A common denominator for all weighted reward contributions divides a fixed power of D_0^{N+4} times the product of the squared denominators of the distinct prices. Its logarithm is $O(NL + NS)$. The same denominator argument applies to payments, cap-multiplier contributions, bound multipliers, and the squared local dual terms; a difference of two prices introduces denominators already in this product. The sum contains only polynomially many compact-pair terms, and their magnitudes add at most polynomial factors and the bounded-height price magnitudes. This gives the stated height. Forward and backward dynamic programming on the reached graph then uses polynomially many operations on polynomial-height integers. $\square$

This is a conservative implementation bound, not an assertion that the experimental total rewards attain it. In the implementation, reward and payment contributions are first grouped by effective price before their final sum. This reduces repeated greatest-common-divisor work across unrelated price denominators without changing the exact result. The independent audit reconstructs conditional payments directly from the action/transition records and checks local normal-cone conditions; it does not trust the compiler’s claimed response identity.

For trusted internally generated experiments, the worker explicitly lifts Python’s default decimal integer-string length guard. This affects serialization only, not mathematical arithmetic or input validity. The general compiler does not silently reinterpret floating-point inputs or accept arbitrary external executable data. The worker’s total reward can have many more decimal digits than a stored slope, precisely the distinction addressed by the proposition.

EC.5 Behavioral Quotients and Information Accounting

A backward signature at a reached pair consists of its rational action and the ordered tuple of successor-vertex and successor-class indices. Two signatures are equal precisely when the corresponding complete suffix plans are equal. Because the graph is acyclic, successor classes are already known when a vertex is processed. The stored class index may be reused at other public vertices. The update map has the public transition as input and writes the successor's local class before the next action is decoded.

This timing convention is important. A past branch cannot be made available to the next action decoder as an additional uncharged register merely by calling it the last observed edge. Similarly, a remaining promise, past service tier, or retained seed would enlarge the observation model if made separately available. The minimal-alphabet theorem fixes these interfaces. It compares controllers with the same read-only program and current public observation, not architectures with different free history channels.

EC.5.1 A strict reduction beyond counting numerical prices

Take a root fixed at zero, two equally likely intermediate vertices, and a common terminal vertex fixed at tier $1/4$. Intermediate rewards are $2x - x^2/2$, their caps are $1/2$ and 1 , payments and discounting are one, and the root promise is $3/4$. Both branch caps bind. The intermediate actions are $1/4$ and $3/4$, so two different prices can reach the terminal vertex. Yet its terminal action is fixed and has no successors. Both prices therefore belong to the same terminal behavioral class, and the entire optimal protocol can use one writable symbol. The exact test records two raw states at that vertex but a minimum alphabet of one. Counting numerical prices alone would overstate necessary memory.

EC.5.2 Randomization and restricted-memory values

The Jensen argument in Theorem 5.1 concerns a randomized controller attaining the *full* optimum. It forces every action at every positive-probability public history to equal the unique optimum almost surely. It does not prove that randomization leaves all lower-alphabet optima unchanged. The frontier in Theorem 5.3 is consequently stated for deterministic encodings. These two results have different conclusions and should not be merged into an unsupported randomized-frontier claim.

EC.6 Heterogeneous Cell Costs and Dispersion

For quadratic terminal reward $f(c) = rc - qc^2/2$ and intermediate cost $h_h(y) = \gamma_h y^2/2$, the cell cost at minimum b_i expands as

$$C(i, j) = \sum_{h=i}^j \pi_h \{rb_h - qb_h^2/2 - rb_i + qb_i^2/2 + \gamma_h b_h^2/2 - \gamma_h b_h b_i + \gamma_h b_i^2/2\}.$$

Prefix sums of π_h , $\pi_h b_h$, $\pi_h b_h^2$, $\pi_h \gamma_h$, $\pi_h \gamma_h b_h$, and $\pi_h \gamma_h b_h^2$ therefore evaluate a cell in constant arithmetic time. With general convex cost functions, the segmentation recurrence still applies after cell costs are computed; the quadratic prefix-sum preprocessing bound is not assigned to an arbitrary function oracle.

The finite enumeration test does not assume contiguous cells. It enumerates all set partitions for branch counts two through seven, computes each cell at its smallest cap, and compares the best value at every cell count with the ordered dynamic program. Two cost families are used: heterogeneous quadratic costs with quadratic terminal reward, and heterogeneous cubic costs with a strictly increasing piecewise-concave terminal reward. Probabilities are proportional to branch indices rather than equal. In total, 2,308 arbitrary partitions are checked exactly.

For the dispersion test, probabilities are again unequal, the mean cap is $1/2$, and centered branch indices define d_j . Four rational positive dispersion levels are tested for each branch count. The resulting 84 strict loss comparisons complement, but do not establish, the analytic monotonicity proof. The limiting alphabet jump at zero dispersion follows from the optimal terminal actions, not from a numerical precision threshold.

EC.7 Complete Endpoint-Normal Splitting

The effective interval is the intersection of four half-spaces on one coordinate: physical lower and upper bounds and corridor lower and upper bounds. At a lower endpoint, its outward lower normal is generated by any original lower inequality attaining the maximum lower bound; at an upper endpoint, the upper normal is generated by an inequality attaining the minimum upper bound. At a singleton, both normal directions are present, even if supplied by different original inequalities. Thus their nonnegative sum is the entire required interval normal cone.

For example, let the physical upper bound be $1/2$ and let the corridor upper bound also equal $1/2$. A positive effective upper multiplier u^+ can be split as θu^+ to the corridor and $(1 - \theta)u^+$ to the physical bound for any $0 \leq \theta \leq 1$. Both original constraints are active, so stationarity and complementarity persist. The resulting supporting cut need not be unique. At zero corridor width, both corridor endpoints coincide; the residual's sign determines a valid orientation without dividing by that width. A physical upper bound coinciding with a corridor lower bound similarly creates a singleton, and the corresponding two opposite normals span every residual sign.

The boundary suite tests physical-first, corridor-first, and equal splitting. It includes ordinary coupled-graph queries, upper and lower physical ties, fixed physical tiers, zero corridor widths, and intersections reduced to one point by different physical and corridor bounds. For each feasible anchor, nearby rational table/width queries are solved and the proposed upper support is evaluated exactly. Infeasible test queries are rejected rather than counted as supporting-cut successes. The recorded 150 anchors and 1,185 inequalities are over feasible queries only.

EC.8 Comparison Algorithms and Reproducibility

EC.8.1 Input families

The binary comparison family has T dates, one root, two vertices at every later date, and uniform binary transitions. Its size is $2T - 1$ public vertices and $2^T - 1$ unfolded occurrences. All payment coefficients and discount factors equal one. Reward coefficients and positive quadratic coefficients are drawn from the fixed rational grids specified in `experiments.py` using seeds 13 and 29. A constant tier $2/5$ is feasible; its root payment is $2T/5$. Each cap equals its remaining constant-tier payment plus a strictly positive rational slack. Thus the generic optimizer and the grid method can construct a feasible starting policy without knowing the optimum.

The broadly capped family uses the preserved rational generator with discount $19/20$, heterogeneous positive payment coefficients, and nonuniform positive transitions. Horizon values 16, 32, and 64 at width four are reported. A related tighter-cap instance uses 128 dates, caps $(8 + v \bmod 5)/10$, and root payment $1/4$; its zero lower policy proves the lower envelope feasible. These cases can have many distinct active cap barriers.

The late-renewal family has width four and discount $19/20$. Most caps equal the remaining maximum physical payment and are redundant. At the penultimate date, the four caps are $1/2, 3/5, 7/10, 4/5$, the local reward coefficient is zero, and the terminal reward coefficient is two. Earlier reward coefficients cycle through $0, 1/2, 1$. All quadratic and payment coefficients equal one. A constant tier $1/4$ is feasible and determines the exact root promise. The heterogeneous penultimate caps bind in the reported optima and generate four distinct required terminal tiers after recombination. This is a deliberately sparse-activation scale family, not a surrogate for a worst-case cap pattern.

The dense-knot family has two vertices per later date, discount and payment coefficients one, quadratic coefficients one, slack caps equal to remaining horizon, and distinct rational local reward coefficients. The formula in the generator separates local clipping thresholds. Its response family exhibits quadratic storage growth while its execution has no active cap memory. Reporting both this family and the late-renewal family avoids conflating execution memory with response density.

EC.8.2 Independent allocation and generic convex optimization

The exact tree baseline stores separate occurrence-weighted responses and subtree caps in the allocation units $z_h = w_h a_v x_h$. It forms its marginal response by evaluating at the union of local and child knots and interpolating affine segments. It never imports the quotient compiler, and it does not memoize public vertices. This is a transparent implementation of a tree marginal-reduction scheme. It is not certified as a line-by-line implementation of Mjelde’s or Tang’s published code, and no strongest-known general laminar running time is attributed to it.

The numerical baseline expands the occurrence decision vector, uses Hessian diagonal entries $w_h q_v$ and linear objective entries $-w_h r_v$, and imposes normalized continuation-cap rows, the root equality, and physical bounds. It uses SciPy 1.17.0 (Virtanen et al., 2020), analytic gradients, an exact sparse diagonal Hessian, and sparse linear Jacobians. The reported protocol sets gradient, step, and barrier tolerances to

10^{-12} , with initial barrier parameter and initial barrier tolerance 10^{-10} , and a maximum of 1,000 iterations. Every returned status, iteration count, primal residual, stationarity residual, matrix size, and absolute value discrepancy is retained. The barrier initialization was tightened in a pilot before the final comparison; the pilot outputs remain available and are not substituted for the final table. OSQP was not available in the execution environment, so no OSQP run or commercial-solver comparison is claimed.

EC.8.3 Public-graph promise discretization

At vertex v , the primal state is the remaining promise y . Successor promises are restricted to multiples of $1/Q$. With uniform binary successors, the expected future promise is their average; the current action is the exact residual $x = y - (y_1 + y_2)/2$. Values are computed backward at each public vertex. A max-plus convolution forms the best future reward for each possible sum of the two successor grid indices, reducing repeated successor-allocation work. The root query uses its stipulated rational promise, not a rounded grid approximation.

The objective comparisons in this baseline use double precision. After a maximizing grid index is chosen, the policy is replayed using exact rational arithmetic: conditional promises, every action bound and cap, the root payment, and the implemented reward are checked. Thus the reported grid policy is exactly feasible, but the numerical maximization is not advertised as a proof of exact optimality even within its discretization. Its loss is measured against the exact continuous optimum. Nested-grid feasibility allows a finer grid to reproduce a coarser policy, and the specific regression values are checked to be nondecreasing.

This is a state-space alternative on the public graph, not an SDDP implementation. The SDDP literature supplies more general iterative approximations and convergence results (Pereira and Pinto, 1991; Girardeau et al., 2015; Dowson, 2020). A single-query demand for an exact rational certificate is a different stopping/output specification from a sampled tolerance benchmark. No relative speed assertion about those unimplemented methods is made. The current experiment compares public-state discretization with finite exact dual closure under explicitly common inputs.

EC.8.4 Timing and memory ledger

The numerical environment is Python 3.13.5, NumPy 2.3.5, and SciPy 1.17.0 on the recorded Linux worker. BLAS/OpenMP thread counts are set to one; workers are sequential, not concurrent. This is a shared provisioned environment, not a dedicated hardware benchmark. Timings exclude common numerical-library imports and use `perf_counter`. Raw runs, not only medians, are stored. An absolute process peak includes imports, input construction, response storage, and other retained objects, so it cannot by itself identify the incremental space cost of a small algorithm.

There are 54 completed comparison workers and 12 completed scale workers. Comparison phases are repeated three times. In the scale workers, the single-query certificate and audit are measured once; all table notes disclose that distinction. Whole-response verification on 35 inherited cases and separate small-instance tests establishes a different audit scope. A pilot whole-response audit of a larger graph and an exploratory tighter-cap 1,021-vertex worker exceeded their imposed overall run limits; these censored

attempts are recorded separately and are not assigned completed runtimes or successful certificates. They caution against treating a graph-sized operation bound as uniformly cheap exact arithmetic.

The 101-query timing in Table 2 reuses the root response and tests payments evenly spaced across its feasible interval. It excludes new policy generation and certificate audit. Neither the numerical inversion times nor the small writable alphabet remove the cost of storing rational read-only tables or recompiling after primitive changes.

EC.8.5 Commands and output integrity

From a clean repository checkout with the declared Python dependencies, the preserved R30 computational package runs `tests.py`, `experiments.py -suite comparisons`, `experiments.py -suite scaling`, and `summarize.py`. R31 adds the independent `tightness_check.py` and a separate reader-build workflow. Saved worker outputs permit resuming a completed R30 specification; an explicit rerun mode removes those caches before measuring anew. The historical source/output manifest records hashes, and R31 does not reinterpret timing variation as a correctness failure.

EC.9 R31 Addendum: Tightness and Memory Accounting

This addendum records implementation-facing consequences of R31 and does not alter the retained R30 benchmark data.

EC.9.1 Closed-form tightness family

For the deterministic chain in the R31 tightness theorem, vertex i has local response

$$x_i(\eta) = [2i - \eta]_{[0,1]}.$$

With redundant continuation caps,

$$R_i(\eta) = \sum_{j=i}^n [2j - \eta]_{[0,1]}.$$

An independent audit can enumerate the two events $2j - 1$ and $2j$, verify that all $2n$ event locations are distinct, construct each suffix sum, and count exactly $n^2 + 2n$ affine response segments. This is a symbolic test family for the theorem rather than empirical evidence for it.

EC.9.2 Three accounting quantities

For a fixed root promise let c_v be the number of minimized behavioral classes reached at public vertex v . R31 reports

$$K_* = \max_v c_v, \quad C_* = \sum_v c_v.$$

The first is the common additional writable alphabet when the current public vertex is supplied freely to the decoder. The second counts reached public-observation/behavior pairs. Neither includes the read-only coefficient representation.

The read-only response layer stores the graph plus the exact piecewise-affine responses. With coefficient height S , its direct representation uses $O((N^2 + M)S)$ bits before any implementation-specific compression. A direct minimized transition table stores one successor-class index for every behavior class and outgoing public edge.

EC.9.3 Randomization convention

The exact-optimum randomized lower bound charges a retained private random seed as writable information and imposes continuation feasibility in conditional expectation at each public history. If private randomization must instead satisfy every cap pathwise, the admissible randomized-controller class is smaller, so the exact lower bound cannot be weakened by that stronger convention.

EC.9.4 Post-compilation root queries

Once the complete root response is sorted by its finite event locations, a new feasible root payment is located by binary search and inverted on one affine segment. Constructing and minimizing the reached transducer, emitting a policy certificate, and auditing that certificate are separate downstream costs.

EC.10 R32 referee-closure concordance

R32 does not replace the R31 theorem package. It adds a compact audit layer that makes the independent R30 referee’s remaining priority, complexity, and accounting questions verifiable from the main manuscript.

Accounting convention. The read-only response representation, combined public-state/class count, and changing writable alphabet answer different implementation questions. The new main-text Proposition EC.12.1 collects the already proved bounds without changing their assumptions. In particular, it does not infer a tight bound for $C_* = \sum_v c_v$ beyond the reached-state bound already implied by the fixed-query price set.

Preservation. All R31 theorem, proof, literature, computational, and negative-evidence files remain unchanged on this branch. R32 is an additive reviewability layer. The independent R30 report remains in branch ancestry, and the R31 response remains a complete record of the substantive mathematical revision that first closed its stated theorem gaps.

Table 1: Independent-R30 objection to R32 main-text anchor.

Referee issue	Main-text anchor	Preserved boundary
Parametric-optimization priority	Section 2, Section EC.12.1, Theorems 3.1–3.4	Generic parametric paths and memoization are predecessor principles
Machine/model minimization	Section 5, Theorem 5.1	Backward minimization is classical; finite reached price structure is model-specific
Tight output size	Theorem 3.2, Proposition EC.12.1	Dense experiments are illustrations, not lower-bound proofs
Minimal-memory interpretation	Theorem 5.1, Proposition EC.12.1	K_* is additional writable alphabet, not total controller complexity
Dynamic-contract policy graphs	Section 2, Section EC.12.1	Policy graphs are not claimed as new
Outside-option dispersion	Proposition 5.4	Ordered mean-preserving radial spread only; no general convex-order claim
Bit complexity	Theorem 3.4	Bound is for the stated coefficient-event compiler on rational inputs
Computational baselines	Section 10	Unmemoized tree code is an independent check, not a claim against normalized memoization

EC.11 Piecewise Response and Randomized-Memory Verification

EC.11.1 Revision lineage and preservation

The additional statements are Theorem 4.1, Proposition 4.2, Theorem 6.1, and Proposition 6.2. The quadratic theorem, its explicit-table lower bound, the deterministic behavioral quotient, and the original frontier remain in the article. The independent review was written against a fixed R30 commit; R31 and R32 already supplied several responses before the present extension. The response document identifies inherited and new changes separately. Earlier source modules, proofs, unfavorable cache results, and computational records are not overwritten.

The R32 three-layer accounting statement is retained below as a concordance. Its response-storage tightness concerns separately materialized vertex tables. Proposition 4.2 adds a different exact representation and does not contradict that output-size count. The piecewise extension replaces the quadratic event budget by the local-piece budget stated in Theorem 4.1. The statement about additional writable symbols is conditional on a freely observed public vertex and a fixed root query; it is not a minimum over all possible observation interfaces.

EC.11.2 Independent calculations and reproducible commands

The new directory contains `exact_compiler.py` and `verify.py`. Running the latter with Python creates `verification.json` and `representation.csv`. Only the Python standard library is required; rational numbers are represented by reduced fractions and floating-point primitive inputs are rejected. The recorded

seed is 20260923. Runtime fields are descriptive and vary by environment; equality tests use no numerical tolerance.

The implementation covers one local piecewise-quadratic coordinate per vertex. The multiple-coordinate statement in Theorem 4.1 follows analytically by summing local responses; it is not represented as a separately exercised multi-coordinate software feature. Input checks enforce positive curvature, nonnegative domain bounds, contiguous pieces, continuity in value, nonincreasing marginal reward across joins, positive payments, and normalized positive transition probabilities. Invalid or infeasible inputs raise errors rather than produce certificates.

The exhaustive KKT reference is not the tree-reduction baseline from the retained study. On a small unfolding it forms the diagonal quadratic objective and all cap and interval rows. For each independent candidate active set it solves two rational right-hand sides, obtaining the optimizer and multipliers as affine functions of price. All inactive inequalities and active multiplier signs delimit the exact candidate interval. The union of valid intervals must cover the complete real price line. Equality with the compiler is checked coefficient by coefficient on the common refinement and by value at finite endpoints. There is no grid approximation in this full-path check. Degenerate solutions can occur in multiple active sets; coincident representations do not constitute different response segments.

EC.11.3 Randomized institution and regression scope

In the lottery construction, the branch is observed before the terminal symbol is sampled, and branch participation is assessed before that private sample. The symbol, and no retained seed, crosses recombination. The terminal decoder is deterministic conditional on its symbol. The middle branch of Proposition 6.2 has mean payment one half, although its high draw is three quarters. This verifies expected participation and explains precisely why the same policy is not feasible under private-draw-by-private-draw participation.

For a general codebook, the concave-envelope formula removes the branchwise encoder optimization, but optimizing all continuous codebook levels remains a separate problem. The two-symbol example is globally solved by the displayed analytic cases; the grid in the test suite is only a reproducible safeguard against formula or transcription errors. No conclusion about arbitrary-alphabet randomized segmentation or general convex-order comparative statics is inferred from this example.

EC.11.4 Evidence that is and is not supplied

The new rational checks validate the piecewise compiler, complete-path agreement on four small instances, the representation tradeoff, and the solved lottery example. They do not report timings for the algorithm of Klimm and Warode (2021), claim numerical superiority over specialized convex solvers, or introduce empirical service data. The explicit flow reduction identifies which parameter and constraints must be translated before such a software comparison would be valid. This separates a theorem-level comparison from an unperformed benchmark.

EC.12 Retained Three-Layer Accounting Concordance

EC.12.1 Referee-facing priority and complexity closure

The residual contribution can be audited without assigning novelty to the classical reductions on which it builds. Table 3 separates the inherited principle from the model-specific statement and records whether the relevant size bound is matched by a worst-case construction.

The table is deliberately asymmetric. Generic parametricity, normalized memoization, deterministic machine minimization, and finite policy graphs are predecessor principles. The paper’s theorem burden is instead on the compact-input quotient, sharp response size, exact binary implementation, and the contractual interpretation of the minimized reached machine.

Proposition EC.12.1 (Three-layer representation accounting). *Under the assumptions of Sections 2–5, the exact representation separates into the following resources.*

1. *Compilation. The complete read-only response has $O(N)$ distinct global knots, $O(N^2 + M)$ stored rational scalars, and polynomial coefficient height and preprocessing bit complexity. The knot and scalar-storage orders are worst-case tight when $M = O(N)$.*
2. *Fixed-query execution. A root promise reaches at most $N + 1$ raw incoming-price labels. After classical behavioral minimization, the combined reached state count is $C_* = \sum_v c_v$, while the minimum additional writable alphabet under the declared public-observation interface is $K_* = \max_v c_v = O(N)$. The $O(N)$ writable-alphabet order is worst-case tight.*
3. *Query reuse. Once the complete response is compiled, a new feasible root promise is inverted with $O(\log(N + 1))$ exact comparisons plus a constant number of rational arithmetic operations; constructing and auditing the reached execution machine for that query remains separately charged.*

These quantities are not interchangeable: K_ is not total controller-state complexity, C_* is not read-only compiler storage, and the root-inversion bound does not include policy emission or certification.*

Proof. Part 1 combines Theorems 3.1, 3.2, and 3.4. Part 2 combines the fixed-query label bound in Theorem 3.1, Theorem 5.1, and Corollary 5.2. Part 3 is Corollary 3.5. The final distinctions follow from the observation architecture in Section 5. $\square$

This accounting also fixes the interpretation of the service result. A public vertex is an observed operating condition; the writable symbol records only history that is not already present in that public condition. If a different implementation makes additional inherited variables freely observable, the behavioral quotient must be recomputed under that interface. Conversely, charging the public state itself to the controller enlarges total state storage but does not alter the conditional lower bound on the extra writable alphabet established here.

EC.13 Global Codebook Optimization: Implementation and Verification

This section documents Theorems 7.2 and 7.4. The complete structural and algorithmic proofs appear in the article; no omitted numerical experiment is being used to supply a proof step.

EC.13.1 Rational implementation concordance

The implementation accepts integers, rational strings, or exact fractions. It rejects binary floating-point inputs, unordered or repeated caps, nonpositive probabilities, probabilities not summing to one, negative intermediate curvature, and terminal parameters outside $r \geq q > 0$. Sorted distinct caps are part of the interface; sorting unorganized observations would first require sorting their associated probabilities and costs together. An arbitrary-order interface would add $O(k \log k)$ rational comparisons.

The six prefix moment arrays evaluate (30) and (32) without summing over an interval at each query. The routine `tail_polynomial` returns the exact coefficients A, B, C of $Az^2 + Bz + C$; `tail` compares all clamped cell minima and saves the attaining final level. The routine `randomized_frontiers` implements (31) using two cost rows and retained predecessor indices, then combines them with terminal costs. A strict improvement stores a reconstructed codebook; ties can keep any incumbent. Duplicate terminal levels are deleted. Budgets greater than the number of branches repeat the zero-loss solution.

The bit bound does not require that the implementation form a common denominator. Every addition, multiplication, and division returns a reduced fraction. Comparisons are exact cross products. Moment arrays contain sums of fixed-degree input products. Edges use products of moment values and cap endpoints. Tail minimizers divide two such coefficients, with a strictly positive divisor on nonterminal cells. Prefix paths add at most k edge costs, and a complete candidate adds one optimized tail. This dependency pattern rules out uncontrolled nested algebraic extensions: all candidates and the reconstructed lottery probabilities are rational. Evaluating final probabilities adds differences and a division of rational codewords and has the same polynomial height bound. The stated conservative S covers both objective values and reconstructed controls, rather than only response coefficients.

For convex nondecreasing intermediate costs that are not quadratic, Lemma 7.1 and the prefix recurrence still apply when terminal reward is quadratic. Each terminal interval instead requires minimizing a strictly convex one-dimensional function. Thus the finite reduction persists with a one-dimensional convex minimization oracle. Exact rational attainment and the particular Turing bound are not transferred to a cost family without an exact representation and such an oracle.

EC.13.2 Independent checks and their logical scope

The new verification script uses three complementary comparisons. First, it enumerates every cap-anchored prefix on small instances and every interval for the continuous highest level. This comparator does not call the production dynamic program or its moment routines. Instead, it evaluates the original branchwise lottery formula at three rational points to recover the interval quadratic, minimizes that quadratic exactly, and compares all budgets. It is exhaustive over the theorem-reduced candidate set; Lemma 7.1, not enumeration alone, justifies reduction from all continuous codebooks.

Second, a separate direct evaluator tests codebooks containing noncap levels on a rational grid. Each sampled codebook has loss at least the computed frontier. These are counterexample-search tests and are not described as exhaustive continuous optimization. Third, controller replay reconstructs every branch's adjacent lottery, checks nonnegative probabilities summing to one, verifies its expected cap and the root promise exactly, and recomputes reward independently of prefix costs. It also reports which realized draws would violate pathwise participation. Deterministic solutions are separately replayed using their highest eligible codewords and checked against the pathwise equality proved in Theorem 7.4.

There are 51 small exact instances, comprising 48 seeded heterogeneous cases, the inherited equiprobable three-branch example, the off-cap example, and a one-branch zero-intermediate-curvature boundary. The tests use heterogeneous probabilities, positive rational terminal curvature, intermediate curvatures including zero, and the allowed boundary $r = q$. They perform 1,932 exhaustive candidate comparisons, 446 randomized/deterministic frontier equalities, 1,073 direct nonanchor-grid comparisons, and 1,346 direct moment equalities. Including the scale family, 255 expected-participation controllers are replayed. Ten invalid-input tests verify that unsupported numerical conventions are rejected.

The scale family has 16, 32, 64, and 128 branches and computes every budget from one through eight. Caps are $b_j = j/(k + 1)$, probabilities are $\pi_j = 2j/[k(k + 1)]$, and intermediate curvature is $(j \bmod 5)/3$. Terminal parameters are $r = 2, q = 1$. These are reproducible synthetic inputs, not a service calibration. Machine-dependent elapsed times and exact losses are recorded in `scaling.csv`; no published specialized solver is being timed. The independent exhaustive comparator is appropriate for validating the global frontier, not for asserting practical superiority over specialized parametric-flow software.

EC.13.3 Preserved evidence

The piecewise-quadratic response compiler and its whole-path KKT comparator are rerun in a temporary directory using their unchanged R33 scripts. Their original output files remain unmodified. The new `inherited_verification.json` records the rerun and the script hashes. All R31–R33 theorem modules, the prior response letters, and earlier favorable and unfavorable numerical records remain on the revision branch. The source-preservation manifest checks the inherited reader modules byte for byte. The new frontier does not reinterpret a quadratic explicit-table lower bound as a lower bound against all shared representations, change the public-observation convention, extend the radial comparative static to convex order, or imply a finite-cut guarantee for continuous outer design.

EC.14 Monge Acceleration: Proof Details and Reproduction

This section expands the implementation and boundary arguments for Theorem 8.1. It preserves the global continuous optimization and cap-anchor proofs of Section 7; an ordered search is used only after its hypotheses have been established.

EC.14.1 Edge factorization and triangular boundaries

For $a \leq b \leq c \leq d$, direct expansion of (30) gives

$$\begin{aligned} & E(a, d) + E(b, c) - E(a, c) - E(b, d) \\ &= \frac{q}{2} \left[(b_d - b_c) \sum_{h=a}^{b-1} \pi_h (b_h - b_a) + (b_b - b_a)(b_d - b_c) \sum_{h=b}^c \pi_h \right. \\ & \quad \left. + (b_b - b_a) \sum_{h=c+1}^d \pi_h (b_d - b_h) \right] \geq 0. \end{aligned}$$

Empty sums are zero. Every term is nonnegative, so the argument includes adjacent anchors, zero gaps arising from coincident indices, and arbitrary positive probabilities. The deterministic identity in the article follows by canceling every cap through c ; monotonicity of f and h_h suffices, without a differentiability argument.

For terminal rows $i < j$ and columns $\ell < t$ with $j \leq \ell$, let x attain $Q(j, \ell)$ and y attain $Q(i, t)$. Compactness gives attainment and $x \leq y$, including adjacent intervals with a shared endpoint. Equation (41) gives

$$Q(i, \ell) + Q(j, t) \leq \Psi_i(x) + \Psi_j(y) \leq \Psi_i(y) + \Psi_j(x) = Q(i, t) + Q(j, \ell).$$

The degenerate last cell $I_k = \{b_k\}$ causes no division by zero; its value is evaluated at that endpoint. The derivative argument is not applied across a discontinuity: the joined Ψ_i and the difference in (41) are continuous at every cap because the term being added there has zero value.

For completeness, suppose a later terminal row had a strictly smaller leftmost minimizing column. Both candidate columns would then be feasible for both rows. Monge uncrossing and the two minimizing inequalities force equality, contradicting leftmost selection in the earlier row. The same exchange applies to prefix rows. There the feasible predecessor columns grow with the endpoint; if the later selected column were smaller, both columns would again be feasible for both rows. Thus no artificial infinity values or unproved total-monotonicity convention for missing entries is required.

EC.14.2 Search, storage, and exact arithmetic

The routine `monotone_minima` processes a middle row, scans only the intersection of its feasible columns with inherited lower and upper bounds, and selects the first attaining column. It recurses to the left with that column as upper bound, and to the right with it as lower bound. Induction on the recursion tree shows that the true leftmost row minimizer remains in every search interval. At one depth the intervals have disjoint interiors, with at most one duplicated boundary per sibling pair. The sum of their lengths is $O(k + 2^d)$ at depth d . Summing over $O(\log(k + 1))$ depths gives $O(k \log(k + 1))$ calls, with $O(\log(k + 1))$ recursion stack.

The randomized routine first obtains all terminal minima. It then builds the anchored prefix layers, using the single reachable endpoint at the first layer rather than querying infeasible entries. It retains two cost rows and one predecessor for each reachable layer/endpoint pair. Combination with the terminal costs is a

linear scan per budget. Only the winning candidate is backtracked, once per budget. This detail prevents an uncharged reconstruction loop from being hidden inside the improved search bound. Codebooks share no uncharged history register: the reconstructed symbols and their probabilities use the same decoder interface as the unaccelerated algorithm.

The deterministic routine uses the analogous prefix search with the quadratic cell cost evaluated by the same six prefix moments. The structural cell Monge inequality is valid more generally, but a constant-work cell oracle is part of the stated complexity interface. Sorting unordered inputs adds $O(k \log k)$ exact comparisons and must carry probabilities and costs with each cap. The implemented interface instead rejects repeated or unsorted caps. It also rejects floating inputs, invalid probability sums, negative curvatures, and noninteger or nonpositive alphabet budgets.

All economic calculations use reduced fractions. Neither search introduces an additional optimization parameter or nested algebraic extension. Terminal minimizers still require only the quadratic coefficient ratio and exact clamping; prefix costs still add at most k edges. Thus the height bound in Theorem 7.2 remains a valid conservative bound for values, controls, comparisons, and reconstructed lottery probabilities. Oracle-call counts in the computational table are not represented as bit-operation counts.

EC.14.3 Evidence and independence

The new suite compares both accelerated frontiers with the unchanged complete-frontier R34 algorithms on 98 exact instances, including heterogeneous probabilities and curvatures, zero intermediate curvature, the allowed $r = q$ boundary, near-endpoint caps, and symmetric tie cases. It records 1,712 frontier equalities and 1,712 expected or pathwise controller replays. An inherited but independently coded exhaustive cap-prefix enumerator, which reconstructs each terminal quadratic from direct branchwise values rather than using prefix moments or either dynamic program, supplies 156 additional frontier equalities over 550 candidates.

Direct branchwise checks establish 1,401 terminal-cell oracle identities. There are 2,092 edge, 3,846 terminal-array, and 2,092 deterministic-cell Monge inequality checks, together with 384 leftmost terminal-argmin comparisons. These finite checks are falsification and implementation tests, not proofs of the universal inequalities. An additional 176 nonanchor-grid codebooks do not improve the exact frontier. Fourteen invalid-input tests and 196 conservative recursion-work checks are recorded. The deterministic and randomized production implementations share the inherited primitive moment evaluator; this dependence is disclosed rather than described as complete implementation independence.

The scale study computes budgets one through eight on 16, 32, 64, 128, 256, 512, and 1,024 branches. The first five sizes are compared on identical inputs with the unchanged exact R34 randomized algorithm, with exact equality of every budget value. The larger sizes are accelerated execution and replay tests; the baseline was not run there. Missing baseline entries are therefore neither estimates nor censored timeouts. The separate pathwise implementation is also timed and replayed. Machine-dependent timings, exact losses, oracle counts, source hashes, and environment metadata are retained. This is an ablation of the newly proved cost structure, not a benchmark of a published parametric-flow package, and the instances are not empirically calibrated service data.

EC.15 Linear Search: Completion, Implementation, and Evidence

This section completes the implementation details of Theorem 8.2. All economic reductions, the continuous terminal minimization, and the moment arrays remain those in Sections 7 and 8. The additional step is an ordinal matrix interface on which every submatrix has ordered row minima. It is not enough to verify only the minima of the original triangular matrix.

EC.15.1 Nested feasible domains and exact ties

For each fixed search, use strictly ordered row and column indices. A terminal row i has feasible columns $\ell \geq i$. A prefix row with endpoint j has feasible predecessor indices $i < j$, after columns that cannot be reached at the preceding layer have been removed. The first prefix layer has only its single initialized predecessor. Subsequent layers restrict the columns to the appropriate suffix of globally reachable predecessor indices. Every production row then has a feasible column.

For a feasible entry, let $K_{rc} = (0, A_{rc}, c)$. On an upper staircase, set $K_{rc} = (1, 0, -c)$ for a missing prefix entry. On a lower staircase, set $K_{rc} = (1, 0, c)$ for a missing suffix entry. Compare keys lexicographically. Column indices make all keys within a row distinct, including equal-cost feasible entries. Feasible entries always precede missing entries. The completion is ordinal: it assigns neither a fabricated rational penalty nor a floating-point infinity to an unavailable contract.

For clarity, we verify the pairwise single-crossing condition in all cases. Fix rows $r < s$ and columns $c < d$ and suppose $K_{rd} < K_{rc}$. When both compared columns are feasible in both rows, this preference means $A_{rd} < A_{rc}$. The feasible Monge inequality yields

$$A_{sd} - A_{sc} \leq A_{rd} - A_{rc} < 0,$$

so the preference persists. The same calculation applies after adding an arbitrary preceding-layer potential to each column. Transposition of the anchored-edge or deterministic-cell array preserves the relevant Monge inequality for endpoint rows and predecessor columns.

In the upper staircase there are three other possibilities. If c is missing and d is feasible, the earlier preference follows from the first component. In a later row either this status persists or both entries become missing, in which case $-d < -c$. If both earlier entries are missing, both later entries are missing, with the same strict preference. If both earlier entries are feasible but one or both later entries become missing, those same two cases apply. A feasible c and missing d cannot occur on this staircase.

On the lower staircase a missing d cannot be preferred to feasible c , and when both are missing the positive column indices prefer c . Thus an earlier preference for d requires both entries to be feasible. Since feasible prefixes expand with row index, both remain feasible in the later row and the finite Monge argument applies. This exhausts the cases. The implication survives deletion of any rows or columns, including deletion of every feasible entry of an original row. It proves total monotonicity with a unique ordinal minimizer in every nonempty submatrix. Only the full production rows must have feasible minima.

A two-column regression explains why the signs matter. Restrict an upper staircase to columns 1 and 2 and rows 2 and 3 with zero finite costs. At row 2 only column 2 is feasible. At row 3 neither is feasible.

Uniform missing values with leftmost ties yield minimizing columns 2 and 1, violating monotonicity. The negative-index prefix keys instead yield columns 2 and 2. This is a diagnostic of the linear-search interface, not a counterexample to the feasible Monge theorem or to the earlier divide-and-conquer algorithm.

EC.15.2 Reduction and interpolation invariants

The function `smawk_minima` implements the classical reduction, recursion, and interpolation procedure. For an ordered row list R and column list C , scan C from left to right while maintaining a stack of at most $|R|$ columns. Compare an incoming column with the stack's last column at the row whose position is the current stack length minus one. Pop the last column precisely when the incoming key is smaller. Append the incoming column if the stack is shorter than the row list.

The reduction preserves a minimizing column for every retained row. Suppose a stack column at zero-based position t is popped by a new column. At witness row R_t the new column is preferred, hence it dominates the popped column at all later rows by single crossing. If $t > 0$, the preceding stack column was preferred to the popped column at R_{t-1} when the latter was appended. By the contrapositive of single crossing it dominates at every earlier row. That predecessor cannot have changed while its successor remained on the stack. Thus the popped column is never uniquely needed; later pops only replace its dominators. At $t = 0$ there are no earlier rows. If the stack is full and an incoming column is not appended, the last stack column beats it at the final witness and consequently at every earlier row. This proves reduction, including missing-only submatrices, without a numerical assumption about the padding. It is the classical matrix reduction, not a claim inferred from tests.

Recursively solve the odd-positioned rows using the reduced column list. For each remaining even-positioned row, scan only between the minimizing columns of its solved neighbors, using the full reduced-list boundary where a neighbor is absent. Total monotonicity places its minimizer in that range. A position map is constructed once per recursion level; the code does not repeatedly perform a linear search for a column's position. Every comparison uses the complete exact key, including tie-breaking indices.

Column reduction uses $O(|R| + |C|)$ comparisons. Interpolation scans ranges whose interiors do not overlap, apart from neighboring boundaries, and hence costs $O(|R| + |C'|)$, where C' is the reduced list. Recursion halves the row count, and after the first reduction the column count is no larger than the preceding row count. Summing over levels gives $O(|R| + |C|)$ comparisons and stored indices. No cache of all feasible matrix entries is constructed. Winner values and terminal levels are recomputed after search; this adds only linear work and is counted in the reported oracle totals.

The implementation uses an explicit key callback and ordered index lists. Its precondition is total monotonicity; it does not attempt an expensive automatic proof for arbitrary supplied cost functions. Invalid or repeated row/column indices are rejected. Empty row sets return an empty result; a nonempty row set with no columns is rejected. The staircase wrapper rejects a missing final minimum, which signals a violated feasibility precondition rather than silently converting a missing entry into a contractual value.

EC.15.3 Arithmetic, storage, and execution accounting

The six prefix-moment arrays require linear preprocessing for already ordered inputs. Each finite terminal query constructs its exact quadratic, clamps the rational stationary point to the interval, and returns its objective and level. The singleton last interval is evaluated directly by the same interface. Each finite prefix or deterministic cell uses constant-many reduced rational operations. Lexicographic padding and index comparisons require $O(\log(k+1))$ bits, which are absorbed in the coefficient-height bound S .

There are one terminal search and at most m program-layer searches. Combining terminal and preceding-prefix costs requires a linear scan per budget; only the winning codebook is reconstructed. All codebooks together have at most $1 + \dots + m$ levels, so reconstruction and output storage are $O(m^2) \subseteq O(mk)$. Backpointers use $O(mk)$ indices, while the active search, two cost rows, and terminal optima use $O(k)$ space. As in the preceding arithmetic model, indices and pointers occupy $O(\log(k+1))$ bits. The conservative Turing bound charges comparisons, integer products, and reduced-rational arithmetic; it does not equate a recorded oracle call with one bit operation. Unordered input additionally requires sorting, with $O(k \log(k+1))$ rational comparisons. Budgets above k append references to the attained zero-loss solution; the theorem is stated for $m \leq k$ so that output length is explicitly charged.

The output per budget is a codebook. In the expected-participation institution, the terminal service draw is an adjacent-codeword lottery and the intermediate service makes the branch's conditional-mean payment equal its cap. Fresh randomness is not retained as an uncharged state. In the pathwise institution, the same branch uses its largest feasible deterministic level; Theorem 7.4 identifies this as the unrestricted randomized pathwise optimum in the stated renewal architecture. Search does not change either observation interface or admissibility definition.

EC.15.4 Independent checks and comparator scope

The R36 suite uses 100 exact rational instances, including one branch, equalities in marginal monotonicity, zero intermediate curvature, highly unequal probabilities, and budgets exceeding the exact alphabet threshold. It records 4,232 objective equalities against the unchanged R34 and R35 algorithms and 2,116 controller replays. An independent comparator enumerates every theorem-permitted anchored prefix and obtains each last-interval quadratic from three direct branchwise evaluations, rather than the production moments or dynamic program. It checks 156 budget objectives across 550 terminal candidates. These are exhaustive checks of the theorem-reduced candidates, not an enumeration of the continuum.

The search-specific checks examine all nonempty row and column subsets for selected small actual and tied staircase matrices, including rows whose feasible columns have all been deleted. There are 43,431 exact row-argmin comparisons, 2,595 nonvacuous single-crossing implications, and 16,273 conservative work-bound checks. Additional checks cover 255 directly reconstructed terminal cells, 402 feasible terminal Monge inequalities, 176 nonanchor grid candidates, two explicit padding regressions, and 17 rejected invalid inputs. Nonanchor grids are falsification tests, not the basis of global optimality.

The same-input study computes all eight requested budgets at each size from 16 through 2,048 branches. Both frontiers agree with the preceding monotone implementation at every size; the quadratic randomized

algorithm is rerun through 256 branches. Larger quadratic cases are unexecuted, not censored timing results. Complete codebooks, exact losses, work counts, actual runtimes, environment metadata, and comparator source hashes are saved. Repeated oracle evaluations count repeatedly. The ordinal comparisons and economic queries are reported separately.

The linear bound is an asymptotic guarantee, not uniform practical dominance. At 16 branches, the new expected-participation search makes 534 economic queries versus 289 for divide-and-conquer; at 2,048 branches it makes 128,966 versus 135,597. The deterministic counts at the latter size are 163,838 versus 153,180, so its crossover has not occurred in this study. These observations are retained, not filtered out. The three versions solve the identical global contractual frontier, but share the economic reduction and moment interface. No comparison with a published external parametric-flow solver or calibrated operational dataset is claimed.

References

- Dowson O (2020) The policy graph decomposition of multistage stochastic programming problems. *Networks* 76(1):3–23. doi:10.1002/net.21932.
- Federgruen A, Groenevelt H (1986) The greedy procedure for resource allocation problems: Necessary and sufficient conditions for optimality. *Operations Research* 34(6):909–918. doi:10.1287/opre.34.6.909.
- Girardeau P, Leclère V, Philpott AB (2015) On the convergence of decomposition methods for multistage stochastic convex programs. *Mathematics of Operations Research* 40(1):130–145. Published online July 16, 2014. doi:10.1287/moor.2014.0664.
- Groenevelt H (1991) Two algorithms for maximizing a separable concave function over a polymatroid feasible region. *European Journal of Operational Research* 54(2):227–236. doi:10.1016/0377-2217(91)90300-K.
- Klimm M, Warode P (2021) Parametric computation of minimum-cost flows with piecewise quadratic costs. *Mathematics of Operations Research* 47(1):812–846. doi:10.1287/moor.2021.1151.
- Pereira MVF, Pinto LMVG (1991) Multi-stage stochastic optimization applied to energy planning. *Mathematical Programming* 52:359–375. doi:10.1007/BF01582895.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.

Table 2: Repeated-query and execution-machine measurements

Vertices	101 root inversions	Machine minimization
253	0.0014	0.0044
509	0.0022	0.0129
1021	0.0043	0.0509
2045	0.0087	0.2075

Seconds, median of three repetitions. The 101 payments are equally spaced across the compiled feasible root interval. Only inversion is included; emitting and auditing 101 different policies is not included. Machine minimization is for the original single query.

Table 3: Priority, scope, and tightness map for the residual contribution.

Object	Inherited principle	Statement used in this paper	Status
Unfolded optimum	Laminar separable resource allocation; scalar marginal allocation	Proposition 2.1 is an exact reduction and is treated as predecessor structure	Classical
Complete parametric response	Piecewise-affine parametric paths and output-sensitive parametric optimization	Theorem 3.1: public-DAG occurrence quotient, at most $3N$ global knots, $O(N^2 + M)$ response storage	Upper bound
Response size	Standard worst-case/output-size analysis	Theorem 3.2: $\Omega(N)$ distinct knots and $\Omega(N^2)$ stored affine segments on polynomial-size rational inputs	Tight in order
Exact preprocessing	Rational arithmetic for structured convex programs	Theorem 3.4: coefficient-event sorting, comparison, crossing, and inversion have polynomial binary complexity	Compiler-specific
Execution reduction	Deterministic output-machine minimization; state aggregation	Theorem 5.1: the contract generates a finite reached price transducer with contiguous price classes	Classical minimization plus contract structure
Writable history	Finite-state implementation accounting	$K_* = \max_v c_v$ under the declared free-public-state interface; Corollary 5.2 gives $K_* = \Theta(N)$ in the worst case	Tight in order
Contractual value	Ordered segmentation / finite-memory allocation	Theorem 5.3 gives the exact deterministic memory-value frontier; Proposition 5.4 gives a strict ordered radial-spread comparative static	Exact for stated family
Policy graph	Finite policy-graph representations in dynamic contracts	The reached graph is an exact fixed-query policy graph for the public-information scalar subclass; policy graphs themselves are not claimed as new	Scope boundary